Partie 3 - Gateway et MQTT

Guide de réalisation et de démonstration - SmartHarness Branche : feat/gateway-mqtt | Date : 29 octobre 2026

1. Mon rôle dans le projet

La partie 3 reçoit les diagnostics des deux ECUs, décode leurs trames CAN et publie les événements en JSON sur MQTT. La décision de panne appartient aux ECUs. Le gateway traduit et transporte cette décision. Node-RED et SQLite seront réalisés par le membre 4 ; leur développement ne fait pas partie de ce livrable.

Le travail comprend deux chemins : une démonstration PC immédiatement utilisable sans matériel, puis un firmware ESP32 avec un mode de simulation CAN et un mode CAN physique. Ce guide décrit les modifications réalisées dans le dépôt et les commandes que tu peux reproduire et présenter.

2. Architecture

ECU A (circuits 1 et 2) + ECU B (circuits 3 et 4) -> trames CAN -> gateway -> MQTT Mosquitto sur PC -> abonnés Node-RED / outil mosquitto_sub -> future base SQLite.

Sur PC, python-can utilise un bus virtuel en mémoire. Tous les participants de ce bus doivent être dans le même processus Python. Lancer un émetteur et un récepteur virtuels dans deux terminaux ne les relie pas. La commande simulator.mqtt_demo rassemble les ECUs et le gateway ; MQTT reste une vraie connexion réseau à Mosquitto.

Sur ESP32, le mode esp32-sim fabrique des trames compatibles toutes les deux secondes. Le mode esp32-can reçoit les vraies trames par le contrôleur TWAI, à 250 kbit/s. Les deux modes utilisent le même décodeur C++ et le même chemin JSON/MQTT.

3. État initial et travail réalisé

Le dépôt contenait le simulateur, les contrats CAN/MQTT et un début de publication MQTT. Les scripts MQTT locaux existaient déjà sans être suivis par Git. Ils ont été conservés, et les points d'entrée recommandés sont désormais simulator.mqtt_demo et gateway.can_mqtt_bridge.

Les ajouts couvrent : dépendance Paho fixée, validation défensive des trames, publication QoS 1, reconnexion automatique, tampon mémoire borné, statut retenu online/offline et Last Will, configuration Mosquitto locale, tests avec un vrai broker, firmware ESP32 à deux modes et workflow GitHub Actions.

Les anciens scripts simulator/simulator-mqtt_demo.py et gateway/simulator-can_sender.py sont des essais historiques. Ne pas utiliser leur lancement en deux processus pour démontrer un échange CAN virtuel. Les tests manuels historiques ne sont pas la suite de validation officielle.

4. Préparer Python sur Windows

Ouvrir PowerShell dans le dossier SmartHarness. Utiliser Python 3.12 ou une version compatible avec les dépendances. Un ancien environnement .venv local pointait vers un interpréteur inaccessible ; un environnement séparé .venv-gateway a donc été créé sans supprimer l'ancien.

```
py -3.12 -m venv .venv-gateway
.\.venv-gateway\Scripts\python.exe -m pip install -r requirements.txt
```

Si py -3.12 ne fonctionne pas, installer Python 3.12 puis reprendre ces commandes. Dans cette session, l'interpréteur Python 3.12 fourni par l'application a servi à créer l'environnement. Les versions projet sont python-can 4.6.1 et paho-mqtt 2.1.0. L'activation de l'environnement n'est pas nécessaire si on utilise son

exécutable directement.

5. Configurer et lancer Mosquitto

Mosquitto était déjà installé dans C:\Program Files\Mosquitto sur ce PC. Sur un autre PC, installer Mosquitto depuis son site officiel. Le fichier broker/mosquitto.conf ouvre le port 1883 sur 127.0.0.1 seulement, autorise la démo anonyme locale et désactive la persistance. Il n'écrase pas la configuration système et n'installe aucun service.

Terminal 1, depuis la racine du dépôt :

```
.\scripts\start-broker.ps1
```

Si la politique PowerShell empêche le script, lancer directement :

```
& 'C:\Program Files\Mosquitto\mosquitto.exe' -c broker/mosquitto.conf -v
```

Garder le terminal ouvert. Ctrl+C arrête ce broker. Si le port 1883 est occupé par le service Mosquitto existant, utiliser ce service pour la démonstration ou choisir un autre port dans une copie de configuration et passer --port au simulateur. Les tests automatisés utilisent un port libre distinct et ne touchent pas au service existant.

6. Observer puis lancer la démonstration

Terminal 2 : s'abonner avant de lancer la simulation, car les événements ne sont pas retenus.

```
& 'C:\Program Files\Mosquitto\mosquitto_sub.exe' -h localhost -p 1883 -t 'smartharness/#' -q 1 -v
```

Terminal 3 : lancer la chaîne PC complète.

```
.\.venv-gateway\Scripts\python.exe -m simulator.mqtt_demo
```

La simulation commence avec quatre circuits normaux : aucun événement. Elle déconnecte les quatre circuits : quatre pannes. Elle les reconnecte : quatre récupérations. L'abonné doit recevoir huit événements au total, répartis entre ECU A et ECU B, ainsi que les changements de statut du gateway.

```
{"ecu":"A","circuit":1,"fault_code":1,"active":true,"voltage_mv":0,"command_on":true,"seq":1}
```

Une récupération utilise fault_code=0, active=false et voltage_mv=3300. Les séquences sont propres à chaque ECU : les pannes ont les séquences 1 et 2, les récupérations 3 et 4. Le gateway n'ajoute ni timestamp ni identifiant de démarrage au contrat existant ; le consommateur peut enregistrer son heure de réception.

7. Contrats CAN et MQTT

Seuls les diagnostics CAN 0x080 (ECU A) et 0x081 (ECU B) sont traités. Les circuits 1 et 2 appartiennent à A ; les circuits 3 et 4 à B. Le format est Classical CAN standard 11 bits, huit octets, version 1.

Octet 0 : circuit. Octet 1 : code de panne. Octets 2-3 : tension en mV, uint16 little-endian. Octet 4 : bit 0 commande ON, bit 1 panne active. Octets 5-6 : séquence uint16 little-endian. Octet 7 : version 1.

Le décodeur rejette les identifiants inconnus, les trames étendues/RTR/erreur/FD, les mauvaises longueurs ou DLC, les versions inconnues, les circuits d'un autre ECU, les bits réservés et les contradictions entre code et état actif. Le code 2 est accepté pour compatibilité avec le contrat, mais ce gateway ne détecte pas lui-même l'intermittence.

Les topics de panne sont smartharness/ecu/a/faults et smartharness/ecu/b/faults. Les sept champs JSON du contrat sont conservés. Les événements sont publiés en QoS 1 et retain=false. smartharness/gateway/status est publié en QoS 1 et retain=true, avec {"status":"online"} ou {"status":"offline"}. Un Last Will annonce offline

lors d'une perte de connexion détectée par le broker.

Le statut concerne le gateway, pas la présence des ECUs : le protocole des heartbeats ECU reste à définir. Exécuter un seul gateway actif pour ce topic de statut partagé.

8. Reconnexion, erreurs et limites

Le client Python utilise la boucle réseau Paho et une reconnexion avec délai exponentiel de 1 à 30 secondes. Le bridge continue de recevoir CAN pendant une coupure MQTT ; Paho garde au maximum 256 publications en mémoire, y compris les messages en vol. Une file pleine est signalée explicitement et l'événement est perdu. Un arrêt du processus perd les messages encore en mémoire.

Le mode de démonstration attend les PUBACK avant de quitter. Le bridge continu publie sans attendre chaque accusé pour ne pas bloquer la lecture CAN. Le suivi des PUBACK utilise le callback on_publish : un objet Paho créé hors connexion peut conserver le code NO_CONN même après livraison, ce qui rend son attente standard trompeuse dans ce cas.

QoS 1 autorise les doublons. Aucun filtrage global n'est appliqué : supprimer un événement uniquement sur seq risquerait de perdre une vraie panne après un redémarrage ECU. Le contrat actuel n'a pas de boot_id. La garantie porte sur la livraison au broker, pas sur l'enregistrement SQLite. La reprise n'est pas une garantie de conservation illimitée ou de livraison exactement une fois.

Le bridge signale les erreurs CAN et s'arrête si l'interface devient inutilisable. Relancer après correction. Une trame invalide est ignorée et journalisée. Le firmware tente la récupération TWAI après bus-off.

9. Utiliser un broker différent et un vrai bus CAN

Le simulateur accepte --host et --port. Le bridge accepte aussi --interface et --channel. Exemple Linux avec une interface SocketCAN déjà configurée :

```
python -m gateway.can_mqtt_bridge --interface socketcan --channel can0 --host 127.0.0.1
```

Sur Windows, choisir une interface python-can adaptée à l'adaptateur CAN réel. Le mode virtual du bridge seul attend des pairs dans son processus : utiliser la démo fournie pour le scénario sans matériel.

Variables Python optionnelles : MQTT_HOST, MQTT_PORT, MQTT_USERNAME, MQTT_PASSWORD et MQTT_CA. La variable MQTT_CA active TLS avec le certificat CA indiqué ; adapter alors le port au broker. Ne pas enregistrer les mots de passe dans Git.

10. Préparer le firmware ESP32

Cible de référence : ESP32 DevKit classique (board esp32dev). PlatformIO fixe la plateforme espressif32 6.10.0 et les bibliothèques AsyncMqttClient 0.9.0 et AsyncTCP 3.5.0. Le client asynchrone permet de publier en QoS 1 et de traiter les PUBACK.

```
.\.venv-gateway\Scripts\python.exe -m pip install platformio==6.1.18
Copy-Item firmware/gateway/include/gateway_config.example.h firmware/gateway/include/gateway_config.h
```

Modifier gateway_config.h : WIFI_SSID, WIFI_PASSWORD, MQTT_HOST (adresse LAN du PC), MQTT_PORT et éventuellement les identifiants MQTT. gateway_config.h est ignoré par Git. Les valeurs CHANGE_ME permettent la compilation, mais aucune connexion Wi-Fi réelle.

Le broker de démo écoute uniquement localhost et ne peut donc pas recevoir l'ESP32. Pour l'étape matérielle, préparer une configuration distincte qui écoute sur l'adresse LAN du PC, ajouter une authentification Mosquitto et autoriser le port dans le pare-feu du réseau privé. Ne pas remplacer MQTT_HOST par localhost sur l'ESP32 :

cela désignerait l'ESP32 lui-même. Le firmware fourni utilise MQTT TCP sans TLS ; réserver son utilisation à un réseau de laboratoire maîtrisé.

Pour préparer le broker LAN, créer un utilisateur (le mot de passe est demandé dans le terminal), puis lancer la configuration fournie. Arrêter le broker local précédent pour libérer son port.

```
& 'C:\Program Files\Mosquitto\mosquitto_passwd.exe' -c broker/passwords gateway
& 'C:\Program Files\Mosquitto\mosquitto.exe' -c broker/mosquitto-lan.conf.example -v
```

Renseigner MQTT_USERNAME="gateway" et le mot de passe choisi dans gateway_config.h. broker/passwords est ignoré par Git. Pour observer les topics, ajouter -u gateway et -P suivi du mot de passe à mosquitto_sub (sur un PC partagé, éviter de laisser cette commande dans l'historique). Dans le pare-feu Windows, autoriser Mosquitto uniquement sur le profil privé utilisé pour le laboratoire. Le mot de passe et le réseau LAN seront testés lorsque la carte sera disponible.

11. Compiler et téléverser

Compiler les deux variantes depuis la racine du projet :

```
.\.venv-gateway\Scripts\python.exe -m platformio run -d firmware/gateway -e esp32-sim -e esp32-can
```

Mode esp32-sim : les trames synthétiques suivent quatre pannes puis quatre récupérations, à raison d'un événement toutes les deux secondes. Le cycle se répète et passe par le même décodeur que le CAN réel. Ce mode nécessite un ESP32 et un Wi-Fi/broker accessibles pour une exécution sur carte ; la démo PC n'en a pas besoin.

```
.\.venv-gateway\Scripts\python.exe -m platformio run -d firmware/gateway -e esp32-sim -t upload
.\.venv-gateway\Scripts\python.exe -m platformio device monitor -b 115200
```

Mode esp32-can : même commande de téléversement avec -e esp32-can. Par défaut TX=GPIO21 et RX=GPIO22, configurables dans gateway_config.h. Il faut un transceiver CAN compatible avec l'ESP32, un GND commun et une terminaison adaptée du bus. TWAI ne remplace pas le transceiver. Confirmer le modèle exact de carte et les broches avant le câblage.

Le firmware garde 64 événements en RAM et n'enlève la tête de file qu'après PUBACK. Une coupure conserve l'événement pour retransmission ; un doublon reste possible. Une file pleine est signalée sur Serial. Un redémarrage de carte efface la file. Les tentatives Wi-Fi sont espacées de 10 secondes et MQTT de 5 secondes.

12. Tester et expliquer les résultats

```
.\.venv-gateway\Scripts\python.exe -m unittest discover -s tests -v
```

La suite initiale de 69 tests passait avant les ajouts MQTT. Les nouveaux tests couvrent les trames rejetées, le routage JSON et QoS, la saturation de file simulée, les quatre pannes et récupérations sur CAN virtuel, la réception via un vrai Mosquitto et la reprise après arrêt/redémarrage du broker. Si Mosquitto n'est pas trouvé, les tests réseau sont marqués skipped ; définir MOSQUITTO_BIN avec son chemin pour les activer.

Le test réseau démarre et arrête son propre broker sur un port libre. Il ne modifie pas l'installation Mosquitto. Le test de reprise vérifie le PUBACK d'un événement placé hors connexion, puis vérifie qu'une publication est de nouveau reçue. Résultat local final : 76 tests réussis, aucun échec ni test ignoré. Le statut online retenu et le statut offline après arrêt propre sont également vérifiés. Le lancement réel de simulator.mqtt_demo avec le fichier broker/mosquitto.conf livré a produit huit événements reçus sur MQTT. Le journal de validation local est livré dans docs/gateway-validation.txt.

Un test C++ autonome vérifie le décodeur et son JSON, dont les 256 combinaisons du champ flags. Son exécution avec g++ est prévue dans GitHub Actions ; ce test natif n'a pas été exécuté localement, faute de compilateur C++ hôte disponible. Le workflow gateway-validation.yml exécute la suite Python avec Mosquitto, ce test C++ et les deux compilations PlatformIO. Un workflow ajouté localement n'est exécuté sur GitHub qu'après publication de la branche.

Les deux compilations locales ont réussi : esp32-sim et esp32-can. Le mode simulé utilise 45 796 octets de RAM et 766 409 octets de programme ; le mode CAN physique utilise 45 852 octets de RAM et 772 321 octets de programme, dans les limites de la cible esp32dev. Le journal des deux variantes est conservé dans docs/gateway-firmware-build.txt. Le nom gateway_config.h évite une collision observée avec un fichier config.h du framework. Une bibliothèque tierce signale un avertissement de dépréciation sur AsyncClient::close(bool).

Les tests réseau PC ne valident pas le Wi-Fi, les timings physiques CAN ou les performances de la carte. Le téléversement et la démonstration sur ESP32 restent à effectuer quand le matériel est disponible.

13. Présentation orale et intégration équipe

Pour présenter : expliquer le rôle du gateway, montrer les deux contrats, démarrer le broker, ouvrir l'abonné, lancer simulator.mqtt_demo, montrer les quatre faults puis les quatre recoveries et enfin exécuter les tests. Expliquer la différence entre bus virtuel Python et vrai réseau MQTT.

Au membre 2 : communiquer les IDs, les huit octets et le débit 250 kbit/s. Au membre 4 : fournir les deux topics de panne, les sept champs JSON et le topic de statut. Au membre 5 : fournir la commande de test et le workflow. Les timestamps de réception et la politique de doublons en base doivent être convenus avec les membres 4 et 5.

Pour préparer la remise GitHub, vérifier git status et sélectionner les fichiers de cette partie sur feat/gateway-mqtt. Le travail de cette session reste local ; aucun commit, push ou merge n'est nécessaire pour exécuter la démonstration.

14. Informations nécessaires plus tard

Aucune information supplémentaire n'est nécessaire pour la démonstration PC. Pour l'essai matériel, il faudra le modèle exact d'ESP32, le transceiver CAN et les broches retenues, puis les paramètres Wi-Fi et l'adresse LAN du PC. Saisir les secrets uniquement dans gateway_config.h local. Pour une publication GitHub, il faudra les droits d'écriture sur le dépôt et la procédure de l'équipe.

15. Références

Dépôt : https://github.com/messak-amine/SmartHarness Contrats du dépôt : contracts/can_protocol.md et contracts/mqtt_protocol.md Client Paho : https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html Mosquitto : https://mosquitto.org/documentation/ ESP32 TWAI : https://docs.espressif.com/projects/esp-idf/en/v4.4.7/esp32/api-reference/peripherals/twai.html AsyncMqttClient : https://github.com/marvinroger/async-mqtt-client